This is another place to be careful; the name of the package you downloaded is typically different from the name of the module. Here, the package is django-debug-toolbar, while the Django app is named debug_toolbar. It is the app's name that goes in `INSTALLED_APPS`.

For simpler third-party apps, this is often all you need to do to get going. The Debug Toolbar isn't one of those simpler apps. For example, it is dependent on other apps and configuration, as explained in the documentation:

- The django.contrib.staticfiles app must be in `INSTALLED_APPS` (which it is by default).
- The `STATIC_URL` value must be configured (which is done to RiffMates in chapter 7).
- The `TEMPLATES` configuration must have the `"APP_DIRS"` key set to `True` (which it is by default).

Follow the installation guide, and you'll be fine. Some third-party libraries install views, and for the view to be useful, it needs to be registered as a route. For the Django Debug Toolbar, you do this by adding a *path* inside urls.py:

```
# RiffMates/RiffMates/urls.py
...
urlpatterns = [
    # ...
    path("__debug__/", include("debug_toolbar.urls")),
]
```

In chapter 2, you wrote your first Django view. When the web server handles a request from the browser, it passes this down to the Django framework. Django maps the URL to a view and calls your code. There is a step in-between though: if you wanted to change the behavior of every single view, rather than write the code over and over, Django has a way of hooking all view calls. This is called *middleware*, named as such because it sits in the middle of a request call. Some third-party libraries come with middleware, and the Debug Toolbar uses its middleware to add information about your views into the request, so it can be displayed by the debug overlay. To register middleware, you add its name to the `MIDDLEWARE` configuration value in settings.py. For the Debug Toolbar, it looks like this:

```
# RiffMates/RiffMates/settings.py
...
MIDDLEWARE = [
    # ...
    "debug_toolbar.middleware.DebugToolbarMiddleware",
    # ...
]
```

The order of declaration inside `MIDDLEWARE` defines the execution order of the middleware code. As each piece of code has an effect on the request, the order is important. Be careful to follow the library's installation instructions on the position of any middleware with respect to its siblings.